

PIXEL ART EDITOR

Project Report

A console-based pixel drawing application developed in C

PROJECT PROFILE

Course	CSE115
Section	4
Project Group	6
Programming Language	C

GROUP MEMBERS

Student Name	Student ID
Anindita Barua Sneha	2622449642
Jubair Pasha	2621036642
Nahidul Islam Anim	2622013042
Md. Samiuzzaman Bhuiyan	2622129042
Lamia Nazmul	2625327042

1 Executive Summary

The Pixel Art Editor is a console-based C application built around a fixed 15 x 15 pixel canvas and a 16-color palette. Each cell stores a numeric color value, where 0 represents a blank cell. The application supports single-pixel drawing, erasing, flood fill, palette selection, image loading, PNG saving, mouse-based coordinate selection, and keyboard shortcuts for core commands.

The project was designed as a practical demonstration of fundamental C programming concepts. Its implementation uses a two-dimensional array for visual data, modular functions for editing and rendering, boundary checking for safe coordinate access, a continuous input-processing loop, and palette mapping for representing loaded RGB images using a fixed set of colors.

1.1 Project Objectives

- Represent the drawable image area using a two-dimensional array with a maximum size of 15 x 15 cells.
- Separate the main operations into reusable modules for initialization, drawing, erasing, filling, rendering, loading, and saving.
- Provide an interactive console interface where palette, tools, status, coordinates, and the current canvas remain visible during editing.
- Support mouse-based coordinate selection while retaining keyboard shortcuts for Save, Load, and Quit operations.
- Apply core C programming techniques - arrays, loops, conditions, structures, input handling, validation, and modular design - in a complete application.

1.2 Application Workflow at a Glance

1. Initialize	2. Select	3. Interact	4. Update	5. Continue / Quit
Canvas + console setup	Color + tool	Mouse / keyboard input	Validate + modify cell	Render again until Q

The workflow is intentionally iterative: each accepted action updates the canvas or editor state, then the display is refreshed so that the result is immediately visible to the user.

2 Development Tools, Header Files & Data Structures

The submitted report identifies standard C libraries, Windows console libraries, compile-time constants, and several structures that support the application. The table below explains why each item is relevant to the editor. The descriptions follow the documented project behavior and identify the role each item plays in the application.

Header file	Why it is needed in this project
<code><stdio.h></code>	Provides standard input/output and file-stream functionality used by console interaction and file operations.
<code><stdlib.h></code>	Provides general-purpose C utilities used by program control and supporting operations.
<code><string.h></code>	Supports string/memory-oriented handling used by interface, path, or data-processing logic where required.
<code><conio.h></code>	Provides console-oriented keyboard input facilities appropriate for an interactive Windows console application.
<code><windows.h></code>	Provides Windows console control, mouse/input configuration, cursor handling, and related system-level functionality.
<code><commdlg.h></code>	Provides Windows common-dialog declarations used for file-selection dialogs when loading or saving files.

2.1 Compile-Time Constants and Console Flags

Definition	Purpose in the editor
<code>MAX_WIDTH = 15</code>	Limits the canvas width so all coordinate calculations stay within the supported grid.
<code>MAX_HEIGHT = 15</code>	Limits the canvas height and defines the maximum number of editable rows.
<code>PALETTE_SIZE = 16</code>	Defines the number of fixed palette colors exposed by the interface.
<code>ENABLE_VIRTUAL_TERMINAL_PROCESSING</code>	Enables terminal behavior required for the colored, structured console presentation.
<code>ENABLE_WRAP_AT_EOL_OUTPUT</code>	Controls console output wrapping so the fixed grid layout remains readable.

2.2 Primary Structures

Structure / data	Role	Why it matters
<code>Pixel</code>	Represents a drawable unit/cell and its color state.	Provides a logical unit for pixel-level editing.
<code>Canvas</code>	Represents the bounded 15 x 15 drawing surface.	Centralizes the grid dimensions and pixel data.
<code>UIState</code>	Stores active interaction state such as selected tool/color.	Keeps menu choices consistent across mouse and keyboard input.
<code>GdiplusStartupInput</code>	Initialization data for the graphics subsystem used by image/file handling.	Supports the graphical file-processing portion described by the report.

Implementation note: The submitted report lists `GdiplusStartupInput` as a structure but does not list its corresponding header in the header-file table. The final source code should be checked to confirm the exact GDI+ include and initialization details.

3 Technical Writing: Core Logic & Functional Modules

The editor separates its behavior into small functional responsibilities. This modular approach makes each operation easier to understand, test, and maintain. Because the supplied report does not include full function signatures, the descriptions below use the documented functional roles rather than inventing function names.

3.1 Canvas Initialization

At startup, the requested canvas dimensions are constrained by the configured maximum values. Nested loops then assign color code 0 to every cell, creating a blank canvas before user interaction begins.

```
for each row
  for each column
    canvas[row][column] = 0
```

3.2 Draw / Set Pixel Logic

The draw operation first validates the selected row and column. If both coordinates fall within the 15 x 15 range, the active palette index is written to that cell. This validation prevents out-of-range memory access and guarantees that every edit remains inside the canvas.

3.3 Erase Logic

Erase uses the same coordinate-validation rule as drawing, but writes 0 instead of a palette color. In this data model, setting the cell to zero is the complete definition of returning a pixel to the blank state.

3.4 Flood-Fill Logic

The Fill tool implements a recursive flood-fill operation. It begins at a selected cell, records that cell's target color, and replaces matching cells with the selected replacement color. It recursively explores the four adjacent directions: up, down, left, and right.

```
if outside canvas -> stop
if current color != target -> stop
if target == replacement -> stop
set current cell = replacement
fill(up); fill(down); fill(left); fill(right)
```

3.5 Color & Palette Logic

The application exposes 16 fixed palette entries. The active color is stored as a palette index, and drawing operations place that numeric index into the chosen cell. When an image is loaded, source RGB values are compared with each palette color and the closest palette entry is selected.

3.6 Boundary Validation

Boundary checks are a recurring safety rule throughout the editor. Before any canvas read or write, the implementation checks that the row and column are within the valid range. This protects the fixed-size array and also makes mouse-coordinate mapping safer.

4 Technical Writing: Interface, Menus, Input & File Operations

4.1 Console Menu and User Controls

Control	Action	Logic
Palette 0-15	Select color	Updates the active palette index in the UI state.
1: DRAW	Select draw tool	A valid canvas click writes the active color to one cell.
2: ERASE	Select erase tool	A valid canvas click resets one cell to color code 0.
3: FILL	Select fill tool	A valid canvas click launches four-direction flood fill.
K: SAVE PNG	Save image	Exports the current canvas representation as a PNG.
L: LOAD IMG	Load image	Opens an image, maps RGB colors to the nearest palette, and places the result on the canvas.
Q: QUIT	Exit application	Ends the interactive loop and terminates the program normally.

4.2 User Interface Rendering

The rendering routine redraws the console after meaningful state changes. The visible layout contains the project title, 16-color palette, tool menu, current status, coordinate labels, and the 15 x 15 grid. Blank cells are represented with spaces, while populated cells are shown using the configured console color representation.

4.3 Mouse Input and Coordinate Mapping

Mouse input is translated into screen coordinates and then classified by location. A click may correspond to a palette item, a tool command, or a canvas cell. The corresponding state variable is updated, or the selected grid cell is modified after coordinate validation. Keyboard shortcuts remain available for the file and quit operations.

4.4 Load Image Logic

The load workflow reads an external image and represents it inside the editor's limited 16-color palette. For each source color, the implementation compares the red, green, and blue components against the palette and chooses the color with the smallest squared RGB distance. This is a nearest-color quantization step.

```
distance = (R-r)^2 + (G-g)^2 + (B-b)^2
choose palette color with minimum distance
```

4.5 Save PNG Logic

The save command converts the current 15 x 15 canvas state into an image representation and writes it as a PNG file. The interface reports the operation status so the user knows whether the requested file action has completed.

4.6 Main Interactive Loop

The program continues cycling through render -> input -> validation -> action -> render until the Quit command changes the running state. This loop is the control structure that turns the individual drawing functions into a complete interactive editor.

```
initialize()
while running:
    render UI and canvas
    input = read mouse / keyboard
    action = classify(input)
    validate action
    apply action
quit normally
```

5 Screenshots & Interface Walkthrough

The following screenshots are taken directly from the submitted project report. They are included with explanatory notes so the technical behavior of the interface is documented rather than presented as images alone.

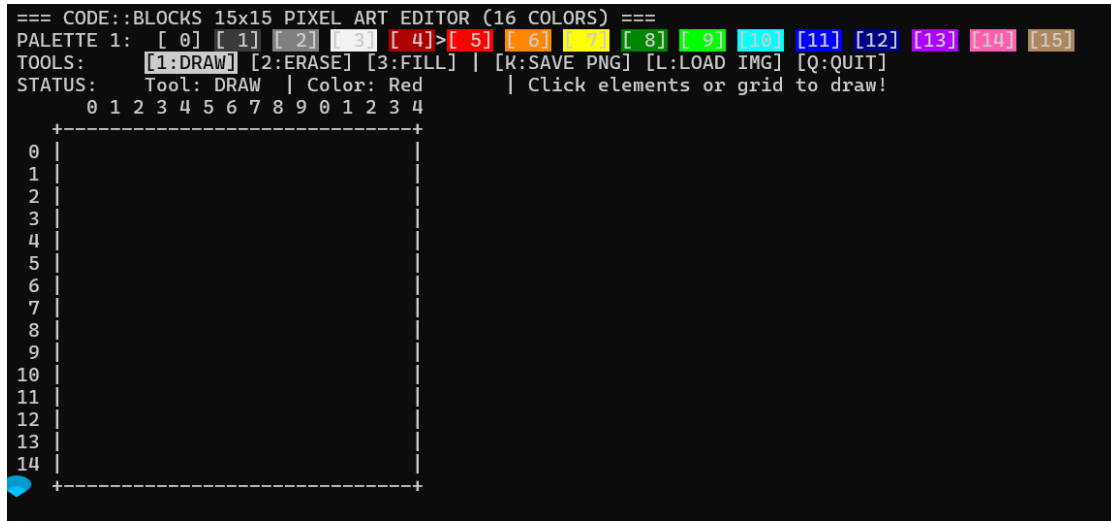


Figure 1. Initial console interface showing the 16-color palette, tool menu, status line, coordinate labels, and blank 15 x 15 canvas.

Figure 1 - What the screen demonstrates

- The title confirms a 15 x 15 pixel editor with a 16-color palette.
- Palette positions 0-15 act as direct color-selection controls.
- The tool line exposes Draw, Erase, Fill, Save, Load, and Quit actions.
- The status line reports the active tool and color and also provides contextual interaction guidance.
- The row and column labels make mouse-to-grid coordinate mapping visible and traceable.



Figure 2. Edited canvas showing populated cells after pixel-level drawing operations.

Figure 2 - What changes after editing

The second screenshot demonstrates that the canvas state is persistent between redraws. Selected cells contain palette colors, the status line records the most recent action, and the rest of the interface remains available for additional edits. This visual feedback confirms the render-after-action workflow described in the implementation.

6 Group Contribution, Testing & Verification

6.1 Group Contribution Table

Feature / Area	Member	Contribution
Environment Setup	Nahidul Islam Anim	Concept, structure, workflow, and canvas coordination.
Color Tool	Anindita Barua Sneha	Selected-pixel color functionality through mouse input.
Draw & Erase Tool	Jubair Pasha	Pixel editing with mouse input and boundary checking.
Fill Tool	Jubair Pasha	Color replacement across matching regions.
User Interface	Anindita Barua Sneha	Interactive menu and tool-selection interface.
Mouse Input	Lamia Nazmul	Mouse coordinate mapping for pixels and controls.
Rendering Tool	Anindita Barua Sneha	Nearest-palette color mapping for loaded images.
Documentation	Md. Samiuzzaman Bhuiyan	Created the final project report.
Testing	Md. Samiuzzaman Bhuiyan	Tested the functionality of the project.

6.2 Functional Test Results

Test case	Expected behavior	Result
Display Canvas	Empty grid shown using spaces	Passed
Coordinate Mapping	Selected pixel coordinates identified through mouse input	Passed
Draw Pixel	Selected pixel updated with chosen color	Passed
Erase Pixel	Selected pixel reset to blank	Passed
Flood Fill	Matching-color region changed to another color	Passed
Color Choice	Active color changed to a new selection	Passed
Load Image	Selected image uploaded to the canvas	Passed
Save Image	Updated image saved to the computer	Passed
Quit Option	Program terminated successfully	Passed

6.3 Verification Coverage

Area	Coverage
Canvas state	Initialization, coordinate mapping, blank/display behavior
Editing tools	Draw, erase, color selection, fill
Interaction	Mouse-based selection and menu controls
File handling	Image loading and PNG saving
Termination	Quit command and normal program exit

According to the submitted test record, every documented scenario passed. The report also states that boundary checks behaved as expected and that the tested draw, erase, fill, load, save, and quit operations completed without reported crashes.

7 Challenges, Design Strengths, Limitations & Future Work

7.1 Challenges and Implemented Responses

Challenge	Implemented response
Processing mouse-based color input	Map screen coordinates to palette, tool, or canvas regions, then update the relevant UI state.
Preventing invalid grid access	Apply explicit boundary conditions before reading or writing a canvas cell.
Keeping the console grid readable	Use spaces for blank pixels, coordinate labels, numeric palette selections, and a consistent layout.
Maintaining a stable interactive loop	Keep rendering, menu handling, and input processing in a continuous cycle until Quit changes the running state.
Representing loaded images with a fixed palette	Calculate RGB distance to each palette entry and choose the nearest available color.

7.2 Design Strengths

- A compact 15 x 15 data model makes the canvas easy to understand, test, and debug.
- Functional separation isolates initialization, editing, filling, rendering, input, and file operations.
- Boundary validation reduces the risk of invalid grid access.
- Mouse interaction plus keyboard shortcuts provides a practical console workflow.
- The fixed palette makes image quantization predictable and keeps the canvas representation simple.

7.3 Current Limitations

The submitted design intentionally remains compact. The drawing area is fixed at 15 x 15 cells and the palette contains only 16 colors, so the editor is less flexible than a full graphics application. The report also notes that more advanced shape drawing and canvas customization are not currently included.

7.4 Recommended Future Improvements

- Add shape tools such as line, rectangle, and other multi-pixel drawing operations.
- Allow users to customize canvas dimensions when creating or importing an image.
- Expand the editing workflow for larger and more flexible drawing tasks.
- Further refine file-handling feedback and validation for a more polished user experience.

8 Conclusion

The Pixel Art Editor demonstrates how core C programming techniques can be assembled into a complete interactive application. The project combines a two-dimensional data model, modular editing logic, console rendering, mouse-based coordinate mapping, image loading, palette quantization, and PNG export. The documented tests passed for all listed scenarios, supporting the conclusion that the implemented feature set worked as intended for the tested cases.

Most importantly, the project provides a clear learning path from basic C constructs to an application with visible state, user interaction, validation, recursion, file handling, and structured program flow.